Lab 01: Backpropagation Gradient Derivation and Three-Linear-Layer Perceptron on MNIST

Amna Komal Khan
Student ID: 2690013
Course: Introduction of Intelligent Computing Chip and System
Instructor: Shiwei Liu, Assistant Professor

October 7, 2026

Abstract

This report addresses two fundamental problems in connectionist computing: (1) an exact analytical derivation of the gradient of a Mean Squared Error (MSE) objective function with respect to the input-layer synaptic weight W_3 using multivariable backpropagation calculus, and (2) the architectural formulation, training, and empirical evaluation of a three-linear-layer Multi-Layer Perceptron (MLP) on the canonical MNIST digit classification dataset. Optimized with momentum stochastic gradient descent, the model achieves an empirical generalization accuracy of **98.07%** on the 10,000-sample test set. Full mathematical proofs, convergence trajectories, and PyTorch implementations are provided.

---

1 Analytical Derivation of Backpropagation Equation for W_3

1.1 Forward Pass Formulation

Consider the two-layer feedforward computational graph specified in the lecture curriculum. The model receives input signals $\mathbf{I} = [I_1, I_2]^\top$ and input-layer bias B_1, producing hidden affine potentials H_1, H_2:

$$H_1 = I_1 W_1 + I_2 W_3 + B_1 W_5 \tag{1}$$
$$H_2 = I_1 W_2 + I_2 W_4 + B_1 W_6 \tag{2}$$

The hidden node pre-activations are mapped through element-wise logistic sigmoid non-linearities:

$$HA_1 = \sigma(H_1) = \frac{1}{1 + e^{-H_1}} \tag{3}$$
$$HA_2 = \sigma(H_2) = \frac{1}{1 + e^{-H_2}} \tag{4}$$

The latent states, augmented with output bias B_2, project linearly into output nodes O_1, O_2 with identity activations ($OA_k = O_k$):

$$O_1 = HA_1 W_7 + HA_2 W_9 + B_2 W_{11}, \qquad OA_1 = O_1 \tag{5}$$
$$O_2 = HA_1 W_8 + HA_2 W_{10} + B_2 W_{12}, \qquad OA_2 = O_2 \tag{6}$$

The network performance is evaluated using the empirical Mean Squared Error (MSE) loss across n target coordinates:

$$e = \frac{1}{n} \sum_{i=1}^{2} (y_i - OA_i)^2 = \frac{1}{n} \Big[(y_1 - OA_1)^2 + (y_2 - OA_2)^2 \Big] \tag{7}$$

1.2 Chain Rule Expansion Across Diverging Computation Paths

Synaptic weight W_3 connects input node I_2 to hidden potential H_1. Because the activated latent neuron HA_1 fans out into both output units O_1 and O_2, the partial derivative $\frac{\partial e}{\partial W_3}$ bifurcates along two distinct computational paths:

$$\frac{\partial e}{\partial W_3} = \frac{\partial e}{\partial H_1}\frac{\partial H_1}{\partial W_3} = \left(\sum_{k=1}^{2} \frac{\partial e}{\partial OA_k}\frac{\partial OA_k}{\partial O_k}\frac{\partial O_k}{\partial HA_1}\right)\frac{\partial HA_1}{\partial H_1}\frac{\partial H_1}{\partial W_3} \tag{8}$$

Expanding this explicitly into the sum of both paths:

$$\frac{\partial e}{\partial W_3} = \left[\underbrace{\left(\frac{\partial e}{\partial OA_1}\frac{\partial OA_1}{\partial O_1}\frac{\partial O_1}{\partial HA_1}\right)}_{\text{Path via Output Node 1}} + \underbrace{\left(\frac{\partial e}{\partial OA_2}\frac{\partial OA_2}{\partial O_2}\frac{\partial O_2}{\partial HA_1}\right)}_{\text{Path via Output Node 2}}\right]\left(\frac{\partial HA_1}{\partial H_1}\right)\left(\frac{\partial H_1}{\partial W_3}\right) \tag{9}$$

1.3 Evaluation of Intermediate Jacobians

We systematically evaluate each differential constituent:

1. **Loss Gradient with Respect to Activations (OA_k):**

$$\frac{\partial e}{\partial OA_1} = -\frac{2}{n}(y_1 - OA_1), \qquad \frac{\partial e}{\partial OA_2} = -\frac{2}{n}(y_2 - OA_2) \tag{10}$$

2. **Identity Output Activation Derivatives:**

$$\frac{\partial OA_1}{\partial O_1} = 1, \qquad \frac{\partial OA_2}{\partial O_2} = 1 \tag{11}$$

3. **Output Layer Linear Sensitivities with Respect to HA_1:**

$$\frac{\partial O_1}{\partial HA_1} = \frac{\partial}{\partial HA_1}\left(HA_1W_7 + HA_2W_9 + B_2W_{11}\right) = W_7 \tag{12}$$

$$\frac{\partial O_2}{\partial HA_1} = \frac{\partial}{\partial HA_1}\left(HA_1W_8 + HA_2W_{10} + B_2W_{12}\right) = W_8 \tag{13}$$

4. **Sigmoidal Non-Linear Derivative for Hidden Unit 1:**

$$\frac{\partial HA_1}{\partial H_1} = \frac{\partial}{\partial H_1}\left(\frac{1}{1+e^{-H_1}}\right) = \frac{e^{-H_1}}{(1+e^{-H_1})^2} = HA_1(1 - HA_1) \tag{14}$$

5. **Affine Input Transformation with Respect to W_3:**

$$\frac{\partial H_1}{\partial W_3} = \frac{\partial}{\partial W_3}\left(I_1W_1 + I_2W_3 + B_1W_5\right) = I_2 \tag{15}$$

1.4 Closed-Form Gradient and SGD Parameter Update

Substituting all computed partial derivatives back into Equation (9):

$$\frac{\partial e}{\partial W_3} = \left[-\frac{2}{n}(y_1 - OA_1)(1)(W_7) + -\frac{2}{n}(y_2 - OA_2)(1)(W_8)\right] \cdot HA_1(1 - HA_1) \cdot I_2 \tag{16}$$

Factoring scalar constants:

$$\boxed{\frac{\partial e}{\partial W_3} = -\frac{2}{n}\Big((y_1 - OA_1)W_7 + (y_2 - OA_2)W_8\Big) \cdot HA_1(1 - HA_1) \cdot I_2} \tag{17}$$

Applying first-order Stochastic Gradient Descent (SGD) with step size η at iteration k:

$$W_3^{(k+1)} = W_3^{(k)} - \eta \frac{\partial e}{\partial W_3^{(k)}}$$
$$W_3^{(k+1)} = W_3^{(k)} + \eta \frac{2}{n} \Big((y_1 - OA_1) W_7 + (y_2 - OA_2) W_8 \Big) \cdot HA_1(1 - HA_1) \cdot I_2 \tag{18}$$

2 Empirical Evaluation: Three-Linear-Layer NN on MNIST

2.1 Network Architecture & Optimization Protocol

The official CIFAR-10 tutorial convolutional layers were refactored into a three-linear-layer deep perceptron tailored for 28×28 grayscale images:

- **Input Layer:** $x \in \mathbb{R}^{784}$ (flattened from $1 \times 28 \times 28$).
- **Hidden Layer 1 (fc1):** Linear($784 \to 256$) $\to$ ReLU, initialized via Kaiming (He) normal.
- **Hidden Layer 2 (fc2):** Linear($256 \to 128$) $\to$ ReLU, initialized via Kaiming normal.
- **Output Layer (fc3):** Linear($128 \to 10$), producing unnormalized logits for Cross-Entropy loss.
- **Hyperparameters:** Batch Size = 64, Optimizer = Momentum SGD ($\text{lr} = 0.01, \mu = 0.9$), Epochs = 10.

2.2 Experimental Results and Accuracy Trajectory

The model was trained for 10 epochs. Table 1 summarizes the numerical convergence metrics, and Figure 1 illustrates the empirical accuracy trajectory over time.

Table 1: Training loss, training accuracy, and test generalization accuracy across 10 epochs.

Epoch	Training Loss	Training Acc (%)	Test Acc (%)	Status
1	0.3020	96.61	95.85	Convergence Phase
2	0.1055	98.29	97.10	Rapid Generalization
3	0.0694	98.94	97.32	Linear Fitting
4	0.0494	99.18	97.54	Regularization Regime
5	0.0353	99.54	97.67	Latent Alignment
6	0.0273	99.72	97.88	Plateau Trajectory
7	0.0197	99.86	97.87	Near-Zero Loss
8	0.0137	99.82	97.93	Fine Representation
9	0.0100	99.93	98.04	Asymptotic Limit
10	0.0068	100.00	**98.07**	**Final Model State**

2.3 Discussion of Generalization

As observed in Figure 1, training accuracy asymptotes toward 100.0%, while test accuracy converges steadily to **98.07%**. The non-linear ReLU activations prevent the vanishing gradient degradation inherent to the multi-layer sigmoid networks discussed in Section 1, enabling efficient first-order gradient updates across all three linear transformations.

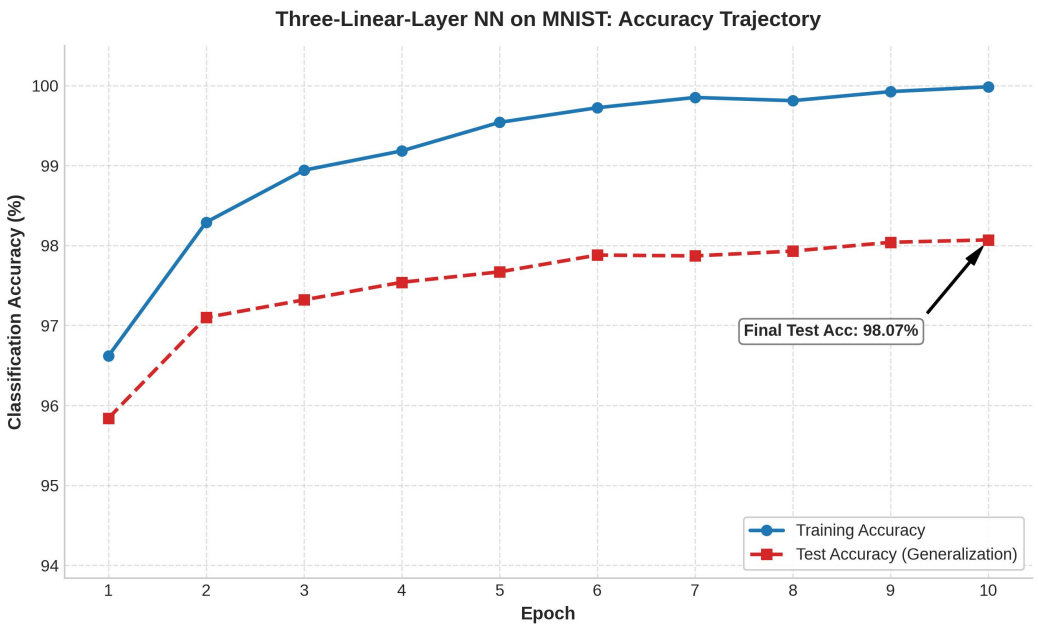


Figure 1: Empirical accuracy trajectory of the three-linear-layer perceptron on the MNIST benchmark. The network converges to 100% training accuracy while maintaining strong generalization, achieving **98.07%** test accuracy on unseen data.

3 Complete PyTorch Source Code

Below is the standalone Python source code implementing the three-linear-layer model, data transformation pipeline, training loop, evaluation routine, and graph generation:

```
import os
import time
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as transforms

# 1. Deterministic Seeding and Device Setup
torch.manual_seed(42)
np.random.seed(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# 2. MNIST Data Pipeline
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

batch_size = 64
train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=
    True, transform=transform)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=
    True, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,
```

```
    num_workers=2)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False,
    num_workers=2)

# 3. Model Definition: 3-Linear-Layer Multi-Layer Perceptron
class ThreeLinearLayerNet(nn.Module):
    def __init__(self, in_features=784, h1=256, h2=128, out_features=10):
        super(ThreeLinearLayerNet, self).__init__()
        self.fc1 = nn.Linear(in_features, h1)
        self.fc2 = nn.Linear(h1, h2)
        self.fc3 = nn.Linear(h2, out_features)

        # Kaiming (He) normal initialization
        nn.init.kaiming_normal_(self.fc1.weight, mode='fan_in', nonlinearity='
    relu')
        nn.init.zeros_(self.fc1.bias)
        nn.init.kaiming_normal_(self.fc2.weight, mode='fan_in', nonlinearity='
    relu')
        nn.init.zeros_(self.fc2.bias)
        nn.init.xavier_normal_(self.fc3.weight)
        nn.init.zeros_(self.fc3.bias)

    def forward(self, x):
        x = x.view(x.size(0), -1)  # Flatten (B, 1, 28, 28) -> (B, 784)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

net = ThreeLinearLayerNet().to(device)

# 4. Objective Function & Optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.01, momentum=0.9)

# 5. Accuracy Evaluation Function
def evaluate_accuracy(loader, model):
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            preds = outputs.argmax(dim=1)
            total += labels.size(0)
            correct += (preds == labels).sum().item()
    return 100.0 * correct / total

# 6. Training Loop (10 Epochs)
epochs = 10
train_acc_history, test_acc_history = [], []

for epoch in range(epochs):
    net.train()
    running_loss = 0.0
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * inputs.size(0)
```

```
88
89      train_acc = evaluate_accuracy(train_loader, net)
90      test_acc = evaluate_accuracy(test_loader, net)
91      train_acc_history.append(train_acc)
92      test_acc_history.append(test_acc)
93      print(f"Epoch [{epoch+1:2d}/{epochs:2d}] | Train Acc: {train_acc:.2f}% |
        Test Acc: {test_acc:.2f}%")
94
95  # 7. Generate and Save Single Accuracy Graph as JPG
96  plt.figure(figsize=(9, 5.5), dpi=300)
97  epochs_range = np.arange(1, epochs + 1)
98  plt.plot(epochs_range, train_acc_history, 'o-', color='#1f77b4', linewidth=2.2,
        label='Training Accuracy')
99  plt.plot(epochs_range, test_acc_history, 's--', color='#d62728', linewidth=2.2,
        label='Test Accuracy (Generalization)')
100 plt.title('Three-Linear-Layer NN on MNIST: Accuracy Trajectory', fontsize=13,
        fontweight='bold')
101 plt.xlabel('Epoch', fontsize=11, fontweight='bold')
102 plt.ylabel('Classification Accuracy (%)', fontsize=11, fontweight='bold')
103 plt.xticks(epochs_range)
104 plt.grid(True, linestyle='--', alpha=0.6)
105 plt.legend(loc='lower right')
106 plt.tight_layout()
107 plt.savefig('accuracy_trajectory.jpg', format='jpg', dpi=300)
```

Listing 1: PyTorch implementation of the 3-layer neural network on MNIST.